

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на разработку веб-приложения «Командный Kanban-менеджер задач»

Тема №44 (в соответствии с комплексным проектным заданием по дисциплине «Веб-дизайн и разработка», 100 академических часов)

1. ОБЩИЕ СВЕДЕНИЯ

1.1. Полное наименование системы: веб-приложение «Командный Kanban-менеджер задач» (рабочее название — «TaskFlow»).

1.2. Условное обозначение: TaskFlow.

1.3. Исполнитель: команда обучающихся 4 курса СПО в составе 2–3 человек. Распределение ролей внутри команды фиксируется в паспорте командного проекта (Приложение А комплексного задания) и матрице ответственности (Приложение Б).

1.5. Сроки выполнения: 100 академических часов в соответствии с календарным планом, приведённым в разделе 4 комплексного проектного задания. Весь объём работ распределяется между участниками команды согласно матрице ответственности.

2. НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ СИСТЕМЫ

2.1. Назначение системы

Веб-приложение «TaskFlow» предназначено для организации командной работы над задачами с использованием методологии Kanban. Система обеспечивает создание рабочих пространств, ведение канбан-досок с настраиваемыми колонками, создание и перемещение задач между колонками, назначение исполнителей, комментирование, ведение истории изменений, а также поиск, фильтрацию и визуализацию статистики по проектам.

Kanban-методология, изначально разработанная для производственных систем, в современном программном обеспечении нашла широкое применение в управлении проектами и задачами. Визуализация рабочего процесса через колонки (например, «To Do», «In Progress», «Done») позволяет команде видеть текущее состояние всех задач без дополнительных совещаний и переписок. Однако для полноценной реализации этой методологии недостаточно просто нарисовать три колонки на бумаге или в таблице — необходим инструмент, который сохраняет порядок задач, фиксирует историю изменений, разграничивает права доступа и позволяет работать команде из разных мест одновременно.

Веб-приложение «TaskFlow» решает именно эту задачу: оно предоставляет команде единое пространство для планирования и отслеживания задач, при этом вся логика — от проверки прав доступа до сохранения порядка карточек — реализуется на сервере, что исключает потерю данных и конфликты при одновременной работе нескольких участников.

2.2. Цели создания системы

Целями создания системы являются:

- автоматизация процессов планирования и отслеживания задач в командах, что снижает количество ручных операций при распределении задач между исполнителями;
- снижение количества ручных операций при распределении задач, включая автоматическое назначение исполнителей, установку сроков и приоритетов;
- обеспечение прозрачности статуса выполнения задач для всех участников команды, что достигается за счёт визуального представления канбан-доски и истории изменений;
- формирование единой истории изменений по каждой задаче, что позволяет отслеживать, кто и когда внёс изменения, а также откатываться к предыдущим состояниям при необходимости;
- предоставление инструментов для анализа загрузки участников и сроков выполнения задач, что реализуется через дашборд с агрегированной статистикой.

2.3. Целевые пользователи

Система ориентирована на несколько категорий пользователей, каждая из которых имеет собственный набор прав и сценариев использования:

Роль	Описание
Гость	Неавторизованный пользователь. Может просматривать публичные доски (при наличии такой настройки) и страницу входа/регистрации.

Роль	Описание
Участник	Авторизованный пользователь, член рабочего пространства. Создает задачи, комментирует, перемещает задачи между колонками, назначает метки.
Администратор пространства	Управляет участниками, досками, колонками, метками. Имеет полный доступ к настройкам пространства.
Системный администратор	Управляет учетными записями, блокирует пользователей, управляет системными справочниками.

Такое разделение ролей соответствует требованиям комплексного задания, где указано, что система должна поддерживать не менее трёх ролей или уровней доступа. В данном случае выделены четыре роли, что даёт запас по функциональности и позволяет гибко настраивать права доступа.

3. ХАРАКТЕРИСТИКА ОБЪЕКТА АВТОМАТИЗАЦИИ

3.1. Предметная область

Предметной областью проекта является управление задачами в командах с использованием Kanban-методологии. В отличие от традиционных систем управления проектами, которые ориентированы на календарное планирование и диаграммы Ганта, Kanban-системы делают акцент на визуализации потока работ и ограничении незавершённых задач. Это позволяет команде быстрее выявлять «бутылочные горлышки» и перераспределять ресурсы.

В рамках учебного проекта предметная область ограничена задачами, связанными с ведением канбан-досок: созданием и перемещением задач, назначением исполнителей и контролем сроков. Финансовые расчёты, интеграция с внешними системами и мобильные приложения в учебную версию не входят — это позволяет сфокусироваться на ключевых аспектах full-stack разработки.

3.2. Проблемы ручного процесса

При организации командной работы без специализированного программного обеспечения команды часто сталкиваются со следующими проблемами:

- задачи фиксируются в разрозненных списках, сообщениях мессенджеров и на бумаге, что затрудняет получение общей картины;
- отсутствует единая история изменений, поэтому невозможно определить, кто и когда изменил статус задачи или назначил исполнителя;
- сложно отслеживать загрузку участников, так как нет сводной информации о количестве активных задач у каждого исполнителя;

- нет прозрачности по статусам задач, что приводит к дублированию работы и потере контекста;
- дублирование информации и потеря контекста при передаче задач между участниками.

Веб-приложение «TaskFlow» устраняет эти проблемы за счёт централизованного хранения данных, серверной проверки прав доступа и визуального представления рабочего процесса.

3.3. Границы проекта

В учебную версию проекта не входят:

- интеграция с внешними системами (GitHub, Jira, Slack и т.д.);
- уведомления по электронной почте и SMS (реализуются только внутренние toast-уведомления);
- мобильные приложения (только адаптивный веб-интерфейс, работающий в браузере);
- платёжные системы и монетизация;
- экспорт в PDF/XLSX (может быть добавлено как бонус за дополнительные баллы).

Явное определение границ проекта важно, поскольку оно позволяет команде сосредоточиться на обязательных требованиях и не тратить время на второстепенные функции.

4. ТРЕБОВАНИЯ К СИСТЕМЕ

4.1. Требования к системе в целом

4.1.1. Требования к структуре и функционированию

Система должна быть построена по клиент-серверной архитектуре: одностраничное приложение (SPA) на React взаимодействует с REST API на Django через HTTP-запросы. Такое разделение позволяет независимо разрабатывать и тестировать frontend и backend, а также обеспечивает возможность повторного использования API для других клиентов (например, мобильных приложений в будущем).

Ролевая модель доступа (RBAC) должна поддерживать четыре роли: гость, участник, администратор пространства, системный администратор. Права доступа проверяются на сервере для каждого запроса — скрывание кнопок на клиенте не является защитой.

Аутентификация выполняется с использованием JWT (JSON Web Tokens) с access и refresh токенами. Access-токен имеет ограниченный срок жизни (например, 1 день), refresh-токен — более длительный (например, 7 дней). Это позволяет снизить риск компрометации токенов при их перехвате.

Адаптивная вёрстка должна обеспечивать корректное отображение на ширинах 360 px (мобильные устройства), 768 px (планшеты) и 1366 px (настольные компьютеры). На мобильной версии не допускается горизонтальная прокрутка основного контента.

Система должна корректно работать в браузерах Chrome, Firefox, Edge (последние версии). Поддержка Internet Explorer не требуется.

4.1.2. Требования к надёжности

Операции перемещения задач и изменения статусов должны выполняться транзакционно. Это означает, что если на каком-то этапе операции возникает ошибка (например, при обновлении порядка задач в

колонке), все изменения откатываются, и данные остаются в согласованном состоянии.

Недопустимые переходы статусов должны блокироваться на сервере. Например, если задача находится в колонке «Done», она не может быть перемещена в колонку «In Progress» без явного разрешения (в зависимости от настроек доски). Сервер хранит граф допустимых переходов и проверяет каждый запрос на соответствие этому графу.

Обработка ошибок должна быть централизованной: пользователь получает понятное сообщение на русском языке, а техническая информация (трассировка стека, детали исключения) записывается в лог сервера и не отображается в интерфейсе.

Резервное копирование базы данных в учебной версии является опциональным, но рекомендуется настроить ежедневное создание дампов для демонстрационного стенда.

4.1.3. Требования к безопасности

Безопасность системы обеспечивается следующими мерами:

- пароли хранятся только в виде стойких хэшей (bcrypt или Argon2). Открытые пароли в базе данных запрещены;
- серверная проверка прав доступа для каждого запроса. Клиентская проверка (скрытие кнопок) является только элементом UX и не заменяет серверную;
- защита от SQL-инъекций через использование ORM (Django ORM) или параметризованных запросов. Строки пользователя не подставляются в SQL вручную;
- экранирование пользовательского контента (XSS). Django по умолчанию экранирует вывод в шаблонах, а для API используется сериализация, которая также предотвращает XSS;

- безопасная загрузка файлов: whitelist типов (PDF, PNG, JPG, ZIP), лимит размера (≤ 10 МБ), серверное имя файла (UUID), запрет исполнения загруженных файлов;
- секреты (ключи JWT, строки подключения к БД) хранятся в переменных окружения (.env) и не попадают в репозиторий. Для разработки предоставляется файл .env.example без реальных значений;
- CORS настроен только для доверенных origins. В production разрешены только известные домены.

4.1.4. Требования к эксплуатации

Запуск системы должен быть воспроизводимым: из чистого окружения по инструкции README. Для этого предоставляется Docker Compose с сервисами frontend, backend и db. Альтернативно — локальный запуск с использованием виртуального окружения Python и npm.

Миграции базы данных должны создавать схему БД автоматически. Seed-данные (демонстрационные записи) позволяют проверить все роли и основные бизнес-сценарии сразу после запуска.

Логирование критических ошибок и важных служебных событий ведётся без записи паролей, токенов и содержимого файлов.

4.2. Требования к функциональным характеристикам

Функциональные требования сформулированы так, чтобы каждый пункт можно было проверить. Ниже приведена таблица с идентификаторами требований, их описанием и критериями приёмки.

ID	Функциональное требование	Критерий приемки
FR-01	Регистрация пользователя по email и паролю	После успешной регистрации создан пользователь; пароль хэширован; вход возможен
FR-02	Авторизация и защищённые маршруты	Неавторизованный запрос к личным данным — 401; доступ к чужим данным — 403
FR-03	Создание рабочего пространства	Администратор создаёт пространство; создатель автоматически становится администратором
FR-04	Приглашение участников в пространство	Администратор добавляет участника по email; участник получает роль «member»
FR-05	Создание и управление досками	Администратор создаёт/удаляет доски; участники видят доски пространства
FR-06	Создание и управление колонками	Администратор создаёт/переименовывает/удаляет колонки; порядок сохраняется
FR-07	CRUD задач	Участник создаёт, редактирует, удаляет задачи; назначение исполнителя, срок, приоритет

ID	Функциональное требование	Критерий приемки
FR-08	Перемещение задач (drag-and-drop)	Перемещение сохраняется на сервере; порядок задач в колонке обновляется
FR-09	Комментирование задач	Участник добавляет комментарии; комментарии видны всем участникам
FR-10	История изменений задачи	Каждое изменение полей (статус, исполнитель, срок) фиксируется в истории
FR-11	Поиск и фильтрация	Поиск по заголовку/описанию; фильтрация по приоритету, исполнителю, меткам
FR-12	Сортировка	Сортировка по сроку, приоритету, дате создания, порядку
FR-13	Пагинация	Списки задач/досок поддерживают пагинацию (20 записей на страницу)
FR-14	Загрузка файлов	Участник прикрепляет файл к задаче; сервер проверяет тип (PDF, PNG, JPG, ZIP) и размер (≤ 10 МБ)
FR-15	Дашборд	Статистика: количество задач по колонкам просроченные, завершённые

ID	Функциональное требование	Критерий приемки
FR-16	Уведомления	Toast-уведомления при успешных/неуспешных операциях
FR-17	Управление метками	Администратор создаёт/удаляет метки; участники назначают метки задачам
FR-18	Администрирование	Системный администратор управляет пользователями (блокировка, смена роли)

Каждое из этих требований соответствует обязательным пунктам раздела 6 комплексного задания. Например, требование FR-08 (перемещение задач) реализует пункт «Статусы и история изменений», а FR-15 (дашборд) — пункт «Дашборд или статистический блок минимум с тремя осмысленными показателями».

4.3. Требования к видам обеспечения

4.3.1. Математическое обеспечение

Специальное математическое обеспечение не требуется, за исключением простых расчётов статистики (количество задач, среднее время выполнения и т.п.), которые выполняются стандартными средствами языка Python и агрегатными функциями базы данных.

4.3.2. Информационное обеспечение

База данных должна содержать не менее восьми связанных таблиц:

- users — учётные записи пользователей;
- workspaces — рабочие пространства;

- `workspace_memberships` — членство пользователей в пространствах с указанием роли;
- `boards` — канбан-доски;
- `columns` — колонки досок;
- `tasks` — задачи;
- `labels` — метки;
- `comments` — комментарии к задачам;
- `task_history` — история изменений задач.

Обязательно наличие связи «многие-ко-многим» — например, между задачами и метками (одна задача может иметь несколько меток, одна метка может быть назначена нескольким задачам).

Миграции должны создавать схему БД автоматически. Seed-данные включают демонстрационные пространства, доски, задачи и учётные записи для всех ролей.

4.3.3. Лингвистическое обеспечение

Интерфейс полностью русифицирован. Сообщения об ошибках формулируются на русском языке и понятны пользователю без технических деталей. Например, вместо «IntegrityError: duplicate key value violates unique constraint» пользователь видит «Пользователь с таким email уже существует».

4.3.4. Программное обеспечение

Рекомендуемый стек:

- **Frontend:** React 18 + Vite + React Router v6 + Axios + Bootstrap + react-beautiful-dnd (или @dnd-kit);
- **Backend:** Django 4.2 + Django REST Framework + djangorestframework-simplejwt + django-cors-headers;

- **База данных:** PostgreSQL 14+ (или SQLite для локальной разработки);
- **Дополнительно:** Docker Compose, Postman (для тестирования API), Git.

Выбор технологий обосновывается в отчёте. Django выбран благодаря встроенной ORM, системе миграций и административной панели, что ускоряет разработку. React обеспечивает компонентный подход и удобство работы с интерактивными элементами (drag-and-drop). PostgreSQL является надёжной реляционной СУБД с поддержкой транзакций и сложных запросов.

4.3.5. Техническое обеспечение

Для развёртывания демонстрационного стенда достаточно:

- сервер: 2 vCPU, 4 ГБ RAM, 20 ГБ SSD (или локальная машина разработчика);
- клиент: любой современный браузер с поддержкой JavaScript.